

บทที่ 5

พื้นฐานของภาษายูเอ็มแอล (UML)

ในบทนี้จะเป็นการกล่าวถึงภาษาที่ใช้การออกแบบซอฟต์แวร์ที่มีชื่อว่า UML (Unified Modeling Language) ภาษา UML เป็นภาษาสัญลักษณ์ที่มีการใช้แผนภาพ และคำอธิบายในการนำเสนอโมเดลของซอฟต์แวร์ ที่ผ่านมานั้นเราได้เคยรู้จักการโมเดลซอฟต์แวร์โดยใช้แผนภาพมาบ้างแล้ว ดังเช่นการพัฒนาซอฟต์แวร์ตามฟังก์ชันนั้นก็จะมีการนำเสนอการทำงานด้วยแผนภาพการไหลของข้อมูลในระบบ (Data Flow Diagram) และการนำเสนอรูปแบบการเก็บข้อมูลด้วยแผนภาพเอนทิตี (Entity)

แต่ในปัจจุบันแนวความคิดของการพัฒนาซอฟต์แวร์ตามฟังก์ชันกำลังจะเปลี่ยนไปเป็นการพัฒนาแบบออบเจกต์โอเรียนเต็ล ซึ่งจากแนวความคิดที่แตกต่างกันของทั้งสองวิธีนี้ส่งผลให้ต้องมีภาษาในการโมเดลซอฟต์แวร์ที่เหมาะสมเกิดขึ้นด้วย และนี่คือที่มาของภาษา UML

5.1 ภาพรวมของ UML

5.1.1 ภาษา UML คืออะไรแล้วทำไมจึงมีความสำคัญ

เนื่องด้วยความสำคัญในการพัฒนาซอฟต์แวร์ที่มีมากขึ้นในปัจจุบัน สิ่งหนึ่งที่มีความจำเป็นและเป็นสิ่งที่ขาดไม่ได้ในการพัฒนาซอฟต์แวร์ก็คือ เทคนิคที่ช่วยอำนวยความสะดวกในการผลิตซอฟต์แวร์ และวิธีที่จะช่วยในการปรับปรุงคุณภาพและลดค่าใช้จ่าย รวมทั้งเวลาในการพัฒนา สำหรับเทคนิคที่มีใช้กันนั้นก็ ได้แก่ เทคโนโลยีคอมพิวเตอร์, การเขียนโปรแกรมแบบวิซวล (Visual Programming) การใช้แพทเทิร์นและเฟรมเวิร์ก (Pattern and Framework) ถึงแม้ว่าเทคนิคเหล่านี้จะสามารถช่วยให้การพัฒนาเป็นไปอย่างมีประสิทธิภาพมากยิ่งขึ้น แต่อุตสาหกรรมการพัฒนาซอฟต์แวร์ก็ยังคงต้องการเทคนิคสำหรับการจัดการความซับซ้อนของซอฟต์แวร์ รวมทั้งต้องการเทคนิคที่สามารถช่วยให้จัดการการพัฒนาซอฟต์แวร์ที่มีขนาดใหญ่ได้ดียิ่งขึ้นด้วยการพัฒนาซอฟต์แวร์ในปัจจุบันนั้นไม่ได้เป็นเพียงการพัฒนาซอฟต์แวร์เพื่อใช้งานบนคอมพิวเตอร์เครื่องเดียว ซอฟต์แวร์ส่วนใหญ่เกือบทั้งหมดมีสถาปัตยกรรมการทำงานเป็นแบบกระจาย ซึ่งการทำงานของซอฟต์แวร์แบบกระจายนี้ผู้พัฒนาจำเป็นต้องพบกับความท้าทายต่าง ๆ ไม่ว่าจะเป็นการทำงานพร้อมๆ กัน (Concurrency), การซ้ำซ้อนข้อมูล (Replication), ความปลอดภัย (Security), การแบ่งภาระงาน (Load Balancing) และความทนทานต่อความเสียหาย (Fault Tolerance)

นอกจากนี้การเติบโตอย่างก้าวกระโดดของ World Wide Web ส่งผลให้แนวความคิดในการใช้งานซอฟต์แวร์เปลี่ยนไป นักพัฒนาแอปพลิเคชันมีทางเลือกของสถาปัตยกรรมมากขึ้นแต่ก็เช่นเดียวกัน ปัญหาที่เกิดขึ้นกับสถาปัตยกรรมก็มีเพิ่มขึ้นตามไปด้วย ด้วยเหตุผลนี้จึงเป็นที่มาของ Unified Modeling Language

(UML) ซึ่งเป็นภาษาที่ได้รับการออกแบบมาเพื่อรองรับกับความท้าทายที่นักพัฒนากำลังเผชิญทั้งในวันนี้และในอนาคต

Unified Modeling Language (UML) เป็นภาษาในการกำหนด (Specifying), แสดงภาพ (Visualizing), สร้างและบันทึกเอกสารเกี่ยวกับสิ่งที่ได้จากการพัฒนาซอฟต์แวร์ รวมทั้งยังมีความสามารถในการโมเดลธุรกิจและระบบอื่น ๆ ที่ไม่ทำงานบนซอฟต์แวร์อีกด้วย ภาษา UML นั้นได้นำเสนอชุดของวิธีปฏิบัติทางด้านวิศวกรรมซอฟต์แวร์ที่ดี ซึ่งวิธีปฏิบัติเหล่านี้จะสามารถช่วยให้การโมเดลและพัฒนาระบบขนาดใหญ่ที่มีความซับซ้อนสามารถทำได้อย่างมีประสิทธิภาพภายใต้เงื่อนไขของเวลาและงบประมาณที่กำหนดไว้

ภาษา UML นั้นได้รับการพัฒนาขึ้นโดยบริษัท Rational Software และผู้ที่เกี่ยวข้องในอุตสาหกรรมนี้ ภาษา UML นั้นเป็นภาษาที่ได้รับการสืบทอดมาจากภาษาในการโมเดลซอฟต์แวร์ที่ประกอบด้วย Booch, OOSE/Jacobson, OMT และวิธีอื่น ๆ รวมกัน โดยมีการนำข้อดีของแต่ละวิธีและแต่ละภาษาประกอบเป็นโครงสร้างหลักของภาษา UML ในปัจจุบันบริษัทและองค์กรหลาย ๆ แห่งได้เริ่มหันมาใช้ภาษา UML ในการกำหนดมาตรฐานสำหรับกระบวนการพัฒนาผลิตภัณฑ์ ซึ่งตัวภาษา UML เองนั้นนอกจากจะเป็นภาษาที่ใช้ในการออกแบบซอฟต์แวร์แล้วยังช่วยกำหนดระเบียบปฏิบัติในการพัฒนาดังเช่น การโมเดลธุรกิจ การจัดการความต้องการ การวิเคราะห์และออกแบบ รวมทั้งการเขียนโปรแกรมและการทดสอบอีกด้วย

5.1.2 ความสำคัญของการโมเดล

ในหลายครั้งนั้นผู้ใช้หรือแม้แต่ผู้พัฒนาซอฟต์แวร์เองมักจะคิดว่าขั้นตอนของการสร้างซอฟต์แวร์ก็คือการเขียนโปรแกรม แต่ความจริงก็ความสำคัญของการพัฒนาซอฟต์แวร์เกือบทั้งหมด (รวมทั้งงบประมาณและเวลา) นั้นจะเกิดขึ้นที่ขั้นตอนของการโมเดลซอฟต์แวร์เป็นเพียงการสร้างพิมพ์เขียวของสิ่งที่ก่อสร้างขึ้น แต่ในการพัฒนาซอฟต์แวร์การโมเดลนั้นจะเป็นขั้นตอนที่มีความสำคัญมากที่สุด การโมเดลซอฟต์แวร์ที่ดีนั้นจะช่วยสร้างรูปแบบในการสื่อสารระหว่างสมาชิกในทีม หรือระหว่างทีมพัฒนาให้มีความเข้าใจที่ตรงและสอดคล้องกัน นอกจากนั้นการโมเดลยังช่วยให้สถาปัตยกรรมของซอฟต์แวร์มีความมั่นคงและยืดหยุ่นอีกด้วย และด้วยความซับซ้อนของซอฟต์แวร์ที่เพิ่มขึ้นอยู่ตลอดเวลา ส่งผลให้อุตสาหกรรมต้องการเทคนิคในการโมเดลซอฟต์แวร์ที่ดี ถึงแม้ว่าจะมีหลายปัจจัยที่เกี่ยวข้องกับความสำเร็จของโครงการพัฒนาซอฟต์แวร์ แต่การมีภาษามาตรฐานสำหรับโมเดลระบบที่ดีก็เป็นสิ่งที่มีความสำคัญไม่แพ้ปัจจัยอื่นเลย

สำหรับภาษาในการโมเดลที่ดีนั้นจะต้องประกอบด้วย

- ส่วนประกอบในโมเดล ซึ่งจะเป็นการนำเสนอแนวความคิดและความหมายหลักของการโมเดล
- สัญลักษณ์ อาจจะเป็นรูปภาพหรือคำอธิบายที่ทำให้ผู้อ่านสามารถเข้าใจส่วนประกอบในโมเดลได้อย่างถูกต้องและครบถ้วน
- แนวทาง เป็นวิธีการใช้สัญลักษณ์ในการอธิบายสิ่งที่จะโมเดล ซึ่งในที่นี้ก็คือซอฟต์แวร์

ยังซอฟต์แวร์มีความซับซ้อนมากขึ้นเท่าใด การโมเดลและการแสดงให้เห็นภาพของระบบก็ยิ่งเป็นสิ่งที่มีความสำคัญตามขึ้นไปด้วย ภาษา UML เป็นภาษาที่ได้รับการกำหนดมาเป็นอย่างดี รวมทั้งเป็นที่ยอมรับอย่างกว้างขวางว่าเป็นภาษาที่ตรงและเหมาะสมกับความต้องการในปัจจุบันมากที่สุด ภาษา UML เป็นภาษาในการโมเดลซอฟต์แวร์ให้เห็นภาพ ซึ่งเป็นทางเลือกหนึ่งในการพัฒนาซอฟต์แวร์ด้วยวิธีออบเจกต์โอเรียนเต็ดและการใช้คอมโพเนนท์เป็นหลัก

5.1.3 เป้าหมายของ UML

เป้าหมายหลักในการออกแบบด้วย UML นั้นประกอบด้วย

- การกำหนดภาษาในการโมเดลที่แสดงให้ผู้ใช้ออฟต์แวร์มองเห็นภาพที่ชัดเจนรวมทั้งสามารถเข้าใจได้ง่าย ทั้งนี้เพื่อเป็นภาษามาตรฐานที่สามารถใช้ในการสื่อสารระหว่างผู้ใช้ออฟต์แวร์กับทีมพัฒนา
- เป็นภาษาที่มีความยืดหยุ่นต่อการขยายการใช้งาน รวมทั้งสนับสนุนกลไกในการกำหนดรายละเอียด เพื่อขยายแนวความคิดหลักให้ตรงกับความต้องการ
- เป็นภาษาในการโมเดลที่ไม่ขึ้นกับภาษาในการโมเดลที่ไม่ขึ้นกับภาษาในการเขียนโปรแกรมและกระบวนการในการพัฒนา
- เป็นภาษาที่ให้โครงสร้างที่มีรูปแบบชัดเจนเพื่อให้สามารถทำความเข้าใจในตัวภาษาสำหรับโมเดลได้ง่าย
- เป็นภาษาที่สนับสนุนการใช้เครื่องมือทางด้านออบเจกต์โอเรียนเต็ดซึ่งมีอยู่ในตลาดอย่างกว้างขวาง
- สนับสนุนแนวความคิดในการพัฒนาซอฟต์แวร์ระดับสูง ดังเช่น แนวความคิดของการทำงานร่วมกัน การใช้เฟรมเวิร์กและคอมโพเนนท์
- เป็นภาษาที่มีวิธีปฏิบัติในการพัฒนาซอฟต์แวร์ที่ถืออยู่ในตัว

ขอบเขตของ UML

Unified Modeling Language (UML) นั้นเป็นภาษาที่ใช้ในการกำหนด สร้าง แสดงภาพและบันทึกเอกสารของสิ่งที่ได้จากกระบวนการพัฒนาซอฟต์แวร์ และเนื่องจากภาษา UML นั้นเป็นภาษาที่ได้รับการกำหนดขึ้นจากการคัดเลือกแนวความคิดที่ดีของวิธี Booch, OMT และ OOSE นั้นส่งผลให้ภาษา UML เป็นภาษาในการโมเดลที่เป็นหนึ่งเดียวมีความง่ายและสามารถใช้งานได้อย่างกว้างขวางสำหรับผู้ที่ใช้ที่คุ้นเคยกับวิธีการพัฒนาอื่นอยู่ก่อนแล้ว

ภาษา UML นั้นมีการกำหนดโครงสร้างและแนวทางในการพัฒนาไว้ในตัว โดยแนวทางที่มีนั้นก็มีพื้นฐานอยู่บนวิธีการสำหรับการจัดการพัฒนาที่มีอยู่ ดังตัวอย่างเช่น ผู้ใช้ภาษา UML สามารถโมเดลซอฟต์แวร์

ได้ทั้งในแง่ของการทำงานร่วมกัน หรือการทำงานของระบบแบบกระจายได้ ซึ่งจากความสามารถเหล่านี้ทำให้ผู้ใช้งานเข้าใจว่า ด้วยคุณสมบัติของ UML ที่มีอยู่เพียงพอกับความต้องการในการจัดการประเด็นต่าง ๆ ในการพัฒนาได้อย่างเพียงพอ

ภาษา UML นั้นเป็นภาษามาตรฐานในการโมเดลซอฟต์แวร์ ไม่ใช่กระบวนการมาตรฐานถึงแม้ว่าภาษา UML นั้นจะต้องนำไปประยุกต์ใช้กับกระบวนการพัฒนาที่มีอยู่ แต่จากประสบการณ์ที่ผ่านมาภาษา UML นั้นได้แสดงให้เห็นแล้วว่า ตัว UML เองนั้นสามารถนำไปประยุกต์ใช้กับองค์กรต่างๆ ที่มีการใช้กระบวนการพัฒนาและที่มีโดเมนของปัญหาแตกต่างกันได้อย่างเหมาะสม ยกตัวอย่างเช่นในการ พัฒนาซอฟต์แวร์ทางด้านบัญชีนั้น กระบวนการในการพัฒนาซอฟต์แวร์ที่เหมาะสมที่สุดนั้นก็คือ การนำคอมโพเนนท์ที่มีอยู่ในท้องตลาดมาประกอบกันเป็นระบบที่สามารถทำงานได้ ซึ่งกระบวนการพัฒนาซอฟต์แวร์แบบนี้ก็จะแตกต่างจากการพัฒนาซอฟต์แวร์ สำหรับควบคุมเครื่องจักรที่ต้องการการพัฒนาโดยมีเงื่อนไขทางเวลาเข้ามาเกี่ยวข้อง แต่ถึงแม้ว่าจะมีความแตกต่างกัน UML ก็สามารถนำไปประยุกต์ใช้กับการพัฒนาทั้งสองแบบได้ ภาษา UML นั้นสนับสนุนการพัฒนาเป็นคอมโพเนนท์ และการพัฒนาโดยมีเงื่อนไขของเวลาเข้ามาเกี่ยวข้อง

นอกจากนั้นภาษา UML ยังเป็นโมเดลพื้นฐานซึ่งมีความหมายอยู่ในตัว และสัญลักษณ์ที่ใช้ก็สามารถเข้าใจได้ง่าย และภาษา UML นั้นยังได้รับการพัฒนาขึ้นโดยมีพื้นฐานอยู่บนการพัฒนาแบบคอมโพเนนท์ การพัฒนาแบบที่มีสถาปัตยกรรมเป็นศูนย์กลาง และการพัฒนาเป็นรอบแบบก้าวหน้า ซึ่งเป็นวิธีการพัฒนาซอฟต์แวร์ที่สามารถช่วยเพิ่มความสำเร็จให้กับโครงการได้มาก

สิ่งที่อยู่นอกเหนือจากขอบเขตของ UML

ถึงแม้ว่าภาษา UML นั้นจะเป็นภาษาในการโมเดลที่ง่ายและเป็นมาตรฐาน แต่ว่าภาษา UML นั้นก็ไม่ได้เป็นภาษาที่ได้รับการออกแบบมาสำหรับทุกๆ สิ่งในการพัฒนาซอฟต์แวร์ และเพื่อให้ภาษา UML เป็นภาษาที่มีความยืดหยุ่น เพื่อให้สามารถนำไปใช้กับระบบซอฟต์แวร์ได้อย่างหลากหลาย ภาษา UML เองจึงได้กำหนดขอบเขตของตัวเองไว้เป็นอย่างดีสำหรับการนำไปใช้กับอุตสาหกรรมและการพัฒนาที่แตกต่างกัน ส่งผลให้สิ่งเหล่านี้ไม่ได้เป็นข้อกำหนดอยู่ในภาษา UML คือ

ภาษาในการเขียนโปรแกรม

ภาษา UML นั้นเป็นภาษาในการ โมเดลที่แสดงให้เห็นภาพของซอฟต์แวร์ ดังนั้นภาษา UML จึงไม่ได้เน้นและเกี่ยวข้องกับภาษาในการเขียนโปรแกรมแบบวิซวล ด้วยเหตุนี้จึงส่งผลให้ภาษา UML นั้นไม่ได้เข้ามาแทน หรือกลายมาเป็นภาษาคู่แข่งสำหรับภาษาในการเขียนโปรแกรมที่มีอยู่ในปัจจุบัน UML นั้นเป็นภาษาสำหรับการแสดงให้เห็นภาพ สำหรับกำหนด สร้าง และจัดบันทึกเอกสารของสิ่งที่ได้จากกระบวนการพัฒนาซอฟต์แวร์ แต่ว่าภาษา UML นั้นไม่ได้เป็นข้อกำหนดให้ผู้ใช้งานต้องเขียนโปรแกรมตามแต่ภาษา UML นั้นก็เป็น

ภาษาที่มีพื้นฐานมาภาษาในการเขียนโปรแกรมแบบออบเจกต์โอเรียนเท็ด ดังนั้นผู้พัฒนาซอฟต์แวร์จึงสามารถใช้ภาษา UML ร่วมกับภาษาในการเขียนโปรแกรมออบเจกต์ได้อย่างหลากหลายและยืดหยุ่น

เครื่องมือ

ภาษาที่เป็นมาตรฐานนั้นจำเป็นต้องมีเครื่องมือและกระบวนการรับรอง เป้าหมายหลักของการกำหนดภาษา UML ให้กลายเป็นมาตรฐานก็เพื่อให้เครื่องมือต่าง ๆ สามารถทำงานร่วมกันได้นั้นจะต้องมีความเกี่ยวข้องในแง่ของความหมายและการกำหนดสัญลักษณ์ตามที่ภาษา UML กำหนดให้ ภาษา UML นั้นจะเป็นตัวกำหนดโมเดลสำหรับแทนความหมาย แต่ไม่ใช่เครื่องมือที่ใช้วาด เก็บ หรือนำโมเดลไปใช้งานจริง แต่ภาษา UML ก็ให้ความสนับสนุนอยู่เบื้องหลังเครื่องมือต่าง ๆ อยู่ด้วยเช่นกัน

กระบวนการ

ถึงแม้ว่าจะมีบริษัทและองค์กรหลายแห่งที่ใช้ภาษา UML เป็นพื้นฐานสำหรับการพัฒนาโครงการซอฟต์แวร์ แต่ว่าบริษัทหรือองค์กรเหล่านี้ก็มีการใช้แผนภาพใน UML สำหรับการนำเสนอการพัฒนาซอฟต์แวร์ที่แตกต่างกัน ภาษา UML นั้นเป็นภาษาที่ได้รับการออกแบบโดยไม่ยึดติดกับกระบวนการ และภาษา UML ก็ไม่ได้มีเป้าหมายที่จะกำหนดกระบวนการมาตรฐานในการพัฒนาซอฟต์แวร์ให้กับนักพัฒนาด้วย

5.2 ส่วนประกอบในภาษา UML

ในภาษา UML นั้นมีการแบ่งส่วนประกอบออกเป็น 3 ส่วนใหญ่ก็คือ สิ่งของ (Thing), ความสัมพันธ์ (Relationship) และแผนภาพ (Diagram) โดยในสิ่งของนั้นจะมีการแบ่งย่อยลงไปเป็นโครงสร้าง (Structural), พฤติกรรม (Behavior), กลุ่ม (Grouping) และคำอธิบายประกอบ (Annotation) ในส่วนของโครงสร้างนั้นจะประกอบด้วย ยูสเคส (Use Case), คลาส (Class), แอกทีฟคลาส (Active Class), อินเทอร์เฟซ (Interface), คอมโพเนนต์ (Component), โครงสร้างการทำงานร่วมกัน (Collaboration) และโหนด (Node) ส่วนพฤติกรรมนั้นประกอบด้วย การติดต่อ (Interaction) และเครื่องจักรสถานะ (State Machine) ส่วนกลุ่มนั้น ประกอบด้วย แพคเกจ (Package), โมเดล (Model), ระบบย่อยและเฟรมเวิร์ก (Pattern and Framework) และส่วนคำอธิบายประกอบนั้นจะประกอบด้วยคำอธิบาย (Note)

สำหรับความสัมพันธ์นั้นก็ประกอบด้วย ความสัมพันธ์ขึ้นอยู่กับ (Dependency) ความสัมพันธ์แบบเกี่ยวข้องกัน (Association) และความสัมพันธ์แบบสืบทอดคุณสมบัติ (Generalization) และแผนภาพนั้นก็ประกอบด้วย แผนภาพยูสเคส แผนภาพคลาส แผนภาพออบเจกต์ แผนภาพแสดงลำดับการทำงานร่วมกัน แผนภาพสถานะ แผนภาพกิจกรรม แผนภาพคอมโพเนนต์ และแผนภาพการติดตั้งใช้งาน

5.2.1 แผนภาพที่มีใน UML

เราอาจจะกล่าวได้ว่าสิ่งที่สำคัญที่สุดในภาษา UML นั่นก็คือแผนภาพแสดงสัญลักษณ์ แต่ทั้งนี้ผู้ที่ใช้ภาษา UML ก็ต้องตระหนักถึงความหมายของโดเมนในภาษา UML ด้วยทั้งนี้ถึงแม้ว่าจะมีการใช้แผนภาพและ

สัญลักษณ์เดียวกัน แต่ถ้าความหมายที่ต้องการนำเสนอมีความแตกต่างกันก็จะเป็นการใช้ภาษา UML ที่ไม่ถูกต้องเช่นเดียวกัน

เราจะได้กล่าวถึงแผนภาพและสัญลักษณ์ที่มีภาษา UML พร้อมทั้งนำเสนอด้วยว่าแผนภาพและสัญลักษณ์ที่มีอยู่นั้นมีความหมายและจะใช้เพื่อนำเสนอสิ่งใดในการพัฒนาซอฟต์แวร์ โดยแผนภาพนั้นจะประกอบด้วย

- แผนภาพยูสเคส ซึ่งจะแสดงฟังก์ชันการทำงานหลักของซอฟต์แวร์
- แผนภาพแสดงการทำงาน (Scenario Diagram) ที่จะแสดงการทำงานในกรณีใดกรณีหนึ่งของซอฟต์แวร์ โดยประกอบด้วย
 - แผนภาพแสดงลำดับการทำงาน (Sequence Diagram) ที่จะเน้นลำดับการทำงานเป็นขั้นตอน
 - แผนภาพแสดงการทำงานร่วมกัน (Collaboration Diagram) ที่จะเน้นแสดงโครงสร้างในการทำงานร่วมกัน
- แผนภาพออบเจกต์ (Object Diagram) แสดงออบเจกต์ที่มีและทำงานอยู่ในซอฟต์แวร์
- แผนภาพคลาส (Class Diagram) แสดงคลาสที่มีอยู่ในซอฟต์แวร์
- แผนภาพแสดงสถานะ (State Diagram) แสดงพฤติกรรมการทำงานของซอฟต์แวร์และส่วนประกอบภายใน
- แผนภาพคอมโพเนนท์ (Component Diagram) แสดงส่วนประกอบและระบบย่อยที่มีอยู่ในซอฟต์แวร์
- แผนภาพการติดตั้งใช้งาน (Deployment Diagram) แสดงการติดตั้งซอฟต์แวร์บนโนดที่มีหน้าที่ประมวลผล
- แผนภาพกิจกรรม (Activity Diagram) แสดงพฤติกรรมการทำงานของซอฟต์แวร์และส่วนประกอบภายใน

5.2.2 แผนภาพยูสเคส

แผนภาพยูสเคสนั้น ในมุมมองของทีมพัฒนาจะเป็นแผนภาพที่ใช้ในการแสดงความต้องการในแง่ของฟังก์ชันการทำงาน (Functional Requirements) ที่สำคัญของระบบ และอาจจะรวมถึงความต้องการที่ไม่เป็นฟังก์ชัน (Nonfunctional Requirement) ที่สำคัญด้วย ส่วนแผนภาพยูสเคส ในมุมมองของผู้ใช้นั้นจะเป็นแผนภาพที่แสดงฟังก์ชันการทำงานหลักของซอฟต์แวร์ ทีมพัฒนาสามารถใช้แผนภาพยูสเคสนี้เป็นเอกสารหลักในการสื่อสารกับผู้ใช้ซอฟต์แวร์ และยังสามารถใช้สื่อสารระหว่างสมาชิกในทีมพัฒนาได้อีกด้วย

สำหรับภายในแผนภาพยูสเคส นั้นจะมีส่วนประกอบที่สำคัญอยู่ 3 ส่วนก็คือ ยูสเคส (Use Case), แอคเตอร์ (Actor) และความสัมพันธ์ระหว่างยูสเคสกับแอคเตอร์ (Relationship)



UseCase

รูปที่ 5.2ก สัญลักษณ์ที่ใช้แทนยูสเคส

สำหรับยูสเคสนั้น จะเป็นตัวบ่งบอกถึงฟังก์ชันที่สำคัญซึ่งระบบสามารถทำได้ โดยการเขียนข้อความภายในยูสเคสนั้น ควรจะสื่อให้เห็นภาพได้ ส่วนยูสเคสนั้นเกี่ยวข้องกับการทำอะไรในซอฟต์แวร์ และเพื่อให้เป็นการแสดงว่ายูสเคสเป็นการแสดงฟังก์ชันการทำงาน ดังนั้นข้อความภายในยูสเคส จึงควรเป็นคำกริยา



Actor

รูปที่ 5.2ข สัญลักษณ์ที่ใช้แทนแอกเตอร์

สำหรับแอกเตอร์นั้นจะเป็นตัวบ่งบอกถึงสิ่งแวดล้อมภายนอกที่เกี่ยวข้องกับซอฟต์แวร์ ดังเช่น ลูกค้า ผู้ดูแลระบบ เป็นต้น และเนื่องจากแอกเตอร์นั้นเป็นสิ่งแวดล้อมดังนั้นแอกเตอร์จึงไม่จำเป็นต้องเป็นมนุษย์เท่านั้น ยังอาจจะเป็นระบบ หรืออุปกรณ์ที่อยู่ภายนอกซอฟต์แวร์ก็ได้ (และยังใช้สัญลักษณ์เดียวกันด้วย) ยกตัวอย่างเช่น สำหรับแอกเตอร์ธนาคารจะมีความเกี่ยวข้องกับซอฟต์แวร์ที่จะพัฒนาขึ้นผ่านยูสเคสสำหรับการโอนเงินผ่านบัตรเครดิต เป็นต้น

สำหรับส่วนประกอบในแผนภาพยูสเคส ที่มีความสำคัญก็คือ ความสัมพันธ์ระหว่างยูสเคสกับแอกเตอร์ซึ่งเราจะใช้เส้นตรงลากจากแอกเตอร์มายังยูสเคสที่เกี่ยวข้อง โดยความสัมพันธ์ระหว่างยูสเคสกับแอกเตอร์นั้นเป็นความสัมพันธ์แบบ Many-To-Many นั้นหมายความว่าแอกเตอร์หนึ่งคนอาจจะเกี่ยวข้องกับหลายๆ ยูสเคส และยูสเคสหนึ่งก็อาจจะเกี่ยวข้องกับแอกเตอร์ได้หลายคน เป็นต้น

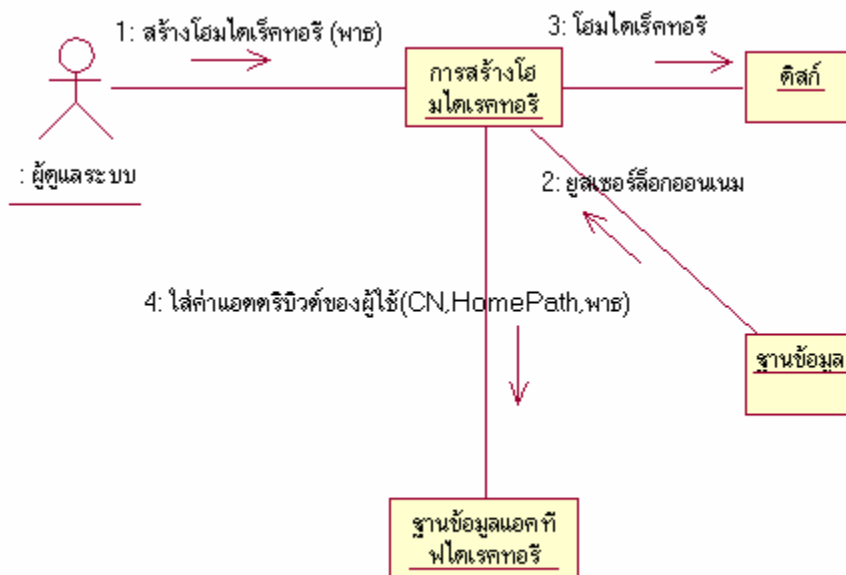
สำหรับยูสเคสภายในแผนภาพนั้นอาจจะมีกำหนดความสัมพันธ์ระหว่างยูสเคสขึ้นมาด้วยโดยที่ความสัมพันธ์ของยูสเคสมีอยู่ 2 ชนิดแตกต่างกันออกไปได้แก่

1. ความสัมพันธ์แบบขยาย (Extend Relationship) ยูสเคสหนึ่งอาจถูกช่วยเหลือโดยการทำงานยูสเคสอื่น เช่น ยูสเคสการใส่รหัสอาจถูกช่วยเหลือโดยยูสเคสคำอธิบายการใส่รหัส สัญลักษณ์ใน UML คือ ลูกศรเส้นประที่ชี้จากยูสเคสแรกไปยังยูสเคสที่ถูกช่วยเหลือหรือถูกขยาย โดยมีคำว่า “extend” อยู่ในเครื่องหมายสเตริโอไทป์ (stereotype) <<extend>> อยู่ที่กึ่งกลางลูกศร
2. ความสัมพันธ์แบบรวม (Include Relationship) ในการทำงานของยูสเคสหนึ่ง ๆ อาจจำเป็นต้องอาศัยการทำงานของยูสเคสอื่น ๆ สำหรับยูสเคสที่ถูกเรียกใช้โดยยูสเคสอื่น สัญลักษณ์ใน UML ของ

ความสัมพันธ์ดังกล่าวคือลูกศรเส้นประชี้ไปยังยูสเคสที่ถูกเรียกใช้หรือถูกรวมไว้ด้วยกัน กล่าวอีกนัยหนึ่งคือยูสเคสที่ถูกยูสเคสอื่น ๆ เรียกใช้งานมากกว่าหนึ่งยูสเคสขึ้นไปมักจะใช้การวาดความสัมพันธ์แบบนี้เข้าช่วยโดยไม่จำเป็นต้องวาดยูสเคสที่ถูกใช้งานซ้ำ ๆ กันหลาย ๆ รูป โดยมีคำว่า “include” อยู่ในเครื่องหมายสเตอริโอไทป์ (stereotype) <<include>> อยู่ที่กึ่งกลางลูกศร

แผนภาพแสดงการทำงานร่วมกัน

แผนภาพแสดงการทำงานร่วมกันนั้นเป็นแผนภาพที่ใช้ในการทำงานของยูสเคสในกรณีหนึ่งคล้ายๆ กับแผนภาพแสดงลำดับการทำงาน แต่จะแตกต่างกันที่วัตถุประสงค์ในการนำเสนอของแผนภาพแสดงลำดับการทำงานร่วมกันนั้นจะเน้นที่การแสดงโครงสร้างในการทำงานภายในยูสเคสมากกว่าการแสดงลำดับ กระนั้นในแผนภาพแสดงการทำงานร่วมกันนี้ก็มีแสดงลำดับการทำงานไว้ด้วยเช่นกัน สำหรับภาพในแผนภาพแสดงการทำงานร่วมกันนี้มีส่วนประกอบเหมือนแผนภาพแสดงลำดับการทำงาน คือ แอคเตอร์ คลาสหรือออบเจกต์ ความสัมพันธ์ระหว่างกัน และข้อความ



รูปที่ 5.2ค ตัวอย่างแผนภาพแสดงการทำงานร่วมกัน (Collaboration Diagram)

จากตัวอย่างแผนภาพแสดงการทำงานร่วมกันนั้น จะเห็นได้ว่าแผนภาพแสดงการทำงานร่วมกันนี้มีรูปแบบการนำเสนอที่แตกต่างจากแผนภาพแสดงลำดับการทำงาน นั่นคือแผนภาพแสดงการทำงานร่วมกันนี้จะนำเสนอให้ผู้พัฒนาและผู้ใช้เห็นภาพโครงสร้างของระบบมากกว่าจะเน้นที่ข้อความที่ไหลอยู่ภายในซอฟต์แวร์

สำหรับการเลือกใช้แผนภาพแสดงลำดับการทำงานหรือแผนภาพแสดงการทำงานร่วมกันนั้นก็จะพิจารณาจากวัตถุประสงค์การใช้งานเป็นหลัก แต่กระนั้นเครื่องมือสำหรับช่วยในการสร้างแผนภาพก็มักจะมีความสามารถในการแปลงระหว่างแผนภาพทั้งสองได้